

TP2

1: FCFS, first come, first server o FIFO

Funcionamiento: Cuando se debe seleccionar un proceso para ejecutar el planificador elige de forma no apropiativa al proceso mas antiguo que se encuentre en la cola de listos, es decir el primero en llegar

El proceso seleccionado retiene el control de la CPU de manera ininterrumpida hasta que completa su rafaga actual

Simulacion de ejemplo:

en $t = 0$ llega Job 1 y toma la CPU

en $t = 1, 2, 3$: llegan Job 2, Job 3 y Job 4 a la cola de listos

en $t = 9$: Job 1 finaliza su rafaga de 9, el siguiente proceso en orden es Job 2, el cual ejecuta sus 5 rafagas

en $t = 14$: Job 2 finaliza. El siguiente es Job 3, inicia su rafaga de 3

en $t = 17$, Job 3 finaliza, Comienza Job 4, que ejecuta 7

en $t = 24$: job 4 finaliza y el lote se completa

2: SJF (Shortest job first/ no apropiativo)

Funcionamiento: Es una politica no apropiativa que selecciona de la cola de listos al proceso que posea la rafaga de cpu mas corta. Si se produce un empate se selecciona en orden de llegada

Simulacion de ejemplo:

en $t = 0$ Solo esta Job 1. Comienza su ejecucion

en $t = 1, 2, 3$: llegan las demas tareas

en $t = 9$, job 1 finaliza. En la cola de listos estan Job 2 (duracion de 5), Job 3 (duracion de 3), Job 4 (duracion de 7). El planificador selecciona a Job 3

en $t = 12$, finaliza Job 3 y comienza Job 2 (el siguiente mas corto)

en $t = 17$ Job 2 finaliza y comienza Job 4

en $t = 24$ Job 4 finaliza su rafaga

3 Round Robin (RR con quantum $q = 4$ y timer variable)

Funcionamiento: Es una politica apropiativa basada en un reloj, donde cada proceso cuenta con un limite de tiempo de ejecucion continua llamado Quantum. Si el proceso no termina antes de que expire el quantum, es expulsado de la CPU, de se le coloca al final de la cola de listos y se selecciona al siguiente de manera circular. En el variante de timer variable el contador se inicializa en Q cada vez que el proceso es asignado a la CPU

Simulacion de ejemplo en $Q = 4$

en $t = 0$ inicia Job 1 (rafaga de 9)

en $t=1,2,3$ llegan Job 2, 3 y 4 a la cola

en $t=4$ se termina el quantum de Job 1 (le faltan 5). Es enviado al final de la cola (actualmente Job 2, 3, 4 y 1). Toma al Job 2 (con 5)

en $t=8$ se termina el quantum de Job 2 (1 pendiente). Es enviado al final de la cola (actualmente job 3,4,1 (pendientes 5), 2 (pendiente 1)). Toma al job 3

en $t=11$ Job 3 finaliza usando solo 3 quantum. La CPU toma al Job 4 (cola job1, job 2, job 4)

en $t=15$ Job 4 es expulsado con 3 pendientes (cola Job 1, Job 2, Job 4). Toma al Job 1 con 5 pendientes

en $t=19$ Job 1 es expulsado con 1 pendiente (cola Job 2, Job 4, Job1). Toma al Job 2 con un pendiente

en $t=20$ Job 2 finaliza (cola Job 4, Job1). Toma al Job 4 con 3 pendientes

en $t=23$ Job 4 finaliza y toma a Job 1

en $t=24$ Job 1 finaliza

4 Algoritmo con uso de propiedades (no apropiativo)

Funcionamiento: Cada proceso tiene asignado un valor numerico que representa su prioridad (por lo general a menor valor, mayor prioridad) Se selecciona el proceso de mayor prioridad esperando en la cola

Simulacion de ejemplo

en $t=0$ inicia Job 1, unico con Prio 3

en $t=9$ Job 1 finaliza. En la cola de listos estan esperando Job 2 (Prio 2), Job 3 (Prio 1) y Job 4 (Prio 2). Toma Job 3

en $t=12$ Job 3 finaliza. En la cola estan Job 2 (Prio 2) y Job 4 (Prio 2), el primero en llegar fue Job 2 entonces se toma ese

en $t=17$ Job 2 finaliza y toma Job 4

en $t=24$ Job 4 finaliza

b. Parametros para su funcionamiento

Varios de estos algoritmos permiten configurar y modificar el comportamiento de la planificacion mediante parametros

1. Round Robin:

Quantum: Es el parametros principal que determina el intervalo de tiempo continuo maximo que un proceso puede utilizar la CPU antes de ser expulsado al final de la cola

2. Planificacion por prioridades

Valor de prioridad: Un parametros numerico asignado a cada prceso que define su urgencia relativa. El OS provee de mecanismos para modificar este parametro por ejemplo con la llamada nice o el comando man nice

Mecanismos de penalizacion o envejecimiento (aging/ penalty): Parametro o regla matematica

que incrementa temporalmente la prioridad de los procesos de baja prioridad a medida que pasa el tiempo para evitar que se queden esperando indefinidamente

c. Algoritmo mas adecuado segun los tipos de procesos y/o SO

FCFS: Es ideal para procesos por lotes en ambientes donde no hay usuarios esperando una respuesta directa en una terminal y se desea maximizar el rendimiento global del procesador sin la sobrecarga de expulsar procesos de forma constante

SJF: Es el algoritmo optimo para entornos batch, ya que matematicamente garantiza minimizar los tiempos de retorno promedio para un lote de procesos, beneficiando sustancialmente a procesos cortos

RR: Es el mas adecuado para sistemas operativos de tiempo compartido e interactivos. Al ser apropiativo asegura equidad y evita que un unico proceso acapare la CPU, permitiendo responder con rapidez las peticiones del usuario

Prioridades: Altamente adecuado para procesos interactivos y de sistema. Permite garantizar que los servicios criticos del OS (que corren de fondo) o acciones prioritarias de usuario se atiendan con preferencia sobre tareas secundarias

d. Ventajas y desventajas de su uso

FCFS: Simple de implementar. NO requiere sobrecarga por expulsion de procesos

Desventajas: No favorece a procesos ligados a entrada/ salida (I/O) haciendolos esperar periodos prolongados. Puede generar tiempos promedio elevados si un proceso largo llega primero

SJF: Minimiza de forma optima los tiempos promedio de esperar y retorno para un lote de trabajos. Pone a los procesos cortos a la cabeza de la cola

Desventajas: Los procesos largos pueden quedarse sin cpu si llegan continuamente procesos cortos. requiere estimar o conocer de antemano el tiempo de la rafaga de CPU futura

RR: Equitativo con la asignacion de cpu.

Desventajas: Si el Q es muy pequeño genera un costo de conmutacion muy alto. Si es muy alto se comporta como un FCFS

Prioridades: Permite al admin o al S dar preferencia a tareas criticas

Desventajas: Procesos de baja prioridad pueden quedarse sin CPU. requiere implementar algoritmos complejos de aging

e. Definiciones de tiempo para un proceso

El TR (tiempo de retorno): Es el tiempo total transcurrido desde el instante en que el proceso llega al sistema (a la cola) hasta que completa su ejecucion

$TR = \text{Tiempo de finalización} - \text{tiempo de llegada}$

El TE (tiempo de espera): es el tiempo total acumulado que el proceso pasa dentro de la cola de listos esperando que se le asigne CPU. Se calcula restando el tiempo de uso efectivo al tiempo de retorno

$TE = TR - T_{CPU}$

f. Definiciones de tiempos promedio para un lote de procesos

Tiempo promedio de retorno (TPR): Es la medida aritmética calculada a partir de la suma de los tiempos individuales de retorno (TR) de cada uno de los procesos que componen el lote

Tiempo promedio de espera (TPE): Es la medida aritmética de los tiempos individuales de espera (TE) calculados para cada proceso del lote

g. Tiempo de respuesta: En procesos interactivos, es el indicador que mide el tiempo transcurrido desde que un usuario o evento realiza una petición en la terminal hasta que el sistema operativo comienza a responder o emite la primera señal de salida de dicha petición. Su principal meta en sistemas de tiempo compartido es responder con la máxima rapidez posible

3.

d. el quantum al ser superior al T_{CPU} y los procesos llegar en el mismo orden se obtiene un resultado muy similar FCFS

4. SJF y SRTF son iguales en este caso (suponiendo que sea el 3 el 5 al que se refiere)

5. Agregar las prioridades y calcular con expulsivas y no expulsivas

a y b son los cálculos

c. La ventaja del algoritmo por propiedades es que permite asegurar los procesos críticos o las interacciones inmediatas por el usuario sean atendidas con preferencia total

Es ideal en OS de propósito general o interactivos como GNU/Linux, donde los servicios del sistema, las llamadas de red y las aplicaciones en primer plano deben responder de inmediato. Cuando podría no ser relevante la prioridad? Cuando todos los procesos tienen el mismo nivel de prioridad carece de sentido ya que añade una sobrecarga de administración innecesaria sin ningún beneficio

6. La inanición es una situación en la que un proceso listo para ejecutarse se ve privado del uso del CPU de manera indefinida, debido a que el planificador selecciona a otros procesos por preferencia o prioridad. El proceso llega y está listo pero es postergado sin poder finalizar su ejecución

b. Los algoritmos que pueden provocarla son los SJF, SRTF, que seleccionan los procesos más cortos posponiendo los largos. Planificación por prioridades, los procesos de baja prioridad van a ser postergados

c. La técnica principal para resolver la inanición es el envejecimiento o aging. Su funcionamiento consiste en permitir que la prioridad de un proceso cambie de manera dinámica a lo largo de su ciclo de vida.

En la planificación por prioridades estándar: El OS incrementa la prioridad de un proceso a medida que este acumula tiempo esperando

En el esquema de colas multinivel: Si un proceso de baja prioridad pasa un intervalo X de tiempo esperando sin ser atendido el sistema lo asciende dinámicamente a una cola de mayor prioridad. Si sigue sin ser atendido tras otro intervalo lo vuelve a ascender. Una vez atendido retorna a su prioridad original

7.

8.

a. El RR distribuye la CPU de manera equitativa, la desventaja es que perjudica notablemente a los procesos ligados a I/O, este debe esperar a que todos los procesos de CPU agoten sus quantums para poder ejecutar otra ráfaga corta

Esto genera una baja utilización de los I/O. Una solución posible es la variante VRR, que crea una cola auxiliar prioritaria para los procesos que retornan de I/O, otorgándoles el tiempo de Q remanente que les falta consumir

b. SRTF es un algoritmo apropiativo que selecciona de la cola al proceso que le quede menos tiempo de ejecución, la desventaja es que provoca inanición en procesos ligados a CPU, quedando así los procesos pesados en cola de manera indefinida

9. El VRR es una variante hecha para mitigar la desventaja planteada en el punto anterior, el VRR posee una cola auxiliar: al retornar de una operación I/O el proceso no vuelve a la cola principal, ingresa a la auxiliar que posee prioridad total de despacho

Para evitar que un proceso abuse de su prioridad, cuando se selecciona un proceso de cola auxiliar se le otorga la CPU únicamente por el tiempo remanente de su quantum original que no llegó a usarse. Si agota el quantum fraccionado sin bloquearse vuelve a la cola principal con un quantum de tamaño estándar

10. Es totalmente posible al esperar un input. El contador de quantum de un proceso solo llega a 0 cuando el proceso es expulsado de forma involuntaria (apropiación) por agotar su tiempo asignado en CPU. Si el proceso es interrumpido de manera voluntario (por petición de I/O o por finalizar su ejecución), la CPU se le quita antes de que el reloj lleve el contador a cero

11.

a. La estimación que más se asemeja a las ráfagas reales es la segunda con $\alpha=0.8$

El proceso representa un patrón de ejecución que cambia de manera drástica, empieza con ráfagas cortas y termina con constantes de 13

La fórmula 1 (Promedio simple) arrastra todo el historial antiguo, por lo que responde de manera extremadamente lenta al cambio

La formula 2 con 0,2 tambien tiene una fuerte inercia historica y no se adapta bien a la subida
En cambio la formula 2 con 0,8 reacciona muy bien al salfo de intensidad, Al ponderar con un 80% el valor real inmediato anterior su estimacion escala rapidamente para luego estabilizarse llegando al final

12.

a. Cola de procesos interactivos: RR

Los procesos interactivos se caracterizan por alternar rafagas cortas de CPU con constantes peticiones de I/O, requieren principalmente tiempos de respuesta rapiudos y equitativos

b. Cola de procesos Batch: FCFS o SJB

Los procesos Batch realizan calculos pesados de larga duracion y no tienen un usuario esperando activamente una respuesta, estos algoritmos no apropiativos eliminan el desperdicio de tiempo del procesador que causan los constantes cambios de contexto

13. resuelto en papel

14.

15.

Estructura de las colas y criterio de ingreso

Niveles de prioridad: se implementan 3 colas de listos, el criterio de ingreso es quen todo proceso nuevo ingresa siempre a la cola de maxima prioridad (Q1). Esto le da a cada proceso la oportunidad inicial de ejecutar rapidamente si se trata de un trabajo corto o acotado por E/S

2 Planificador horizontal (algoritmo para cada cola)

Cada cola administra los procesos mediante su propio algoritmo interno

q1 alta prioridad, se planifica mediante RR con un quantum pequeno (como 4)

q2 prioridad media, se planifica mediante RR con quantum mayor (como 8)

q3 baja prioridad se planifica mediante FCFS o un RR bastante grande

3 mecanismo de realimentacion y penalizacion

para penalizar a los procesos con mayor tempo de ejecucion de cpu

Democion: si un proceso en q1 agota todo su quantum de tiempo sin bloquearse por E/S demuestra ser un proceso acotado por cpu (largo). El sistema lo expulsa y lo manda a q2 y asii sucesivamente

Mantenimiento en alta prioridad: si un proceso libera la cpu voluntariamente antes de agotar su quantum (por ej realizando un E/S), permanece en las colas prioritarias priorizando la interactividad del sistema

4 planificador vertical

Prioridad estricta con apropiacion: el kernel siempre selecciona para ejecutar el proceso que este al frente de la cola no vacia de mayor prioridad

Si un proceso esta ejecutando en Q3 o Q2 y arriba un proceso nuevo a q1 o se desbloquea de E/S, el planificador desaloja de inmediato el proceso actual para atender la cola superior

5 mecanismo para evitar la inanición

dado que un flujo constante de procesos cortos en q1 o q2 podría provocar la inanición de procesos largos en por ejemplo q3, se aplica el aging

si un proceso pasa un tiempo límite T por ejemplo 16 unidades esperando el sistema promueve su prioridad

Una vez el proceso es atendido en la cola superior y ejecuta retorna a su cola original al finalizar su ráfaga o quantum

16.

a. prioridad determinada estáticamente con el método del SJB

Trabajos beneficiados: cortos acotados por CPU y cortos acotados por E/S

Los trabajos cortos tienen prioridad y se atienden de inmediato completando sus ráfagas rápidamente, mientras que los largos se posponen y pueden llegar a sufrir inanición

b. Prioridad dinámica inversamente proporcional al tiempo transcurrido desde la última E/S

Trabajos beneficiados: cortos y largos acotados por E/S

Que la prioridad sea inversamente proporcional al tiempo desde la última E/S significa que cuanto menos tiempo haya pasado desde su última E/S (o cuanto mayor frecuencia de E/S tenga el proceso), mayor será su prioridad. Los procesos acotados por E/S realizan bloqueos constantes para acceder a los dispositivos por lo que mantendrán una prioridad alta de forma dinámica. Esto les permite tomar la CPU rápidamente para ejecutar sus breves ráfagas y volver a disparar operaciones en los dispositivos de E/S, maximizando el uso concurrente del hardware. En cambio los procesos acotados por CPU pasan mucho tiempo continuo ejecutando sin hacer E/S por lo que su prioridad decrece progresivamente

17. Relación entre RR y FIFO al incrementar el quantum

Al tener quantums muy superiores los procesos no llegarán a agotarlo, solo abandonarán la CPU por voluntad propia, la cola de listos pasa a atender a los procesos conforme llegaron y terminamos en una ejecución muy similar a FCFS

18. Glosario importante a tener en cuenta

Printf() imprime texto o variables formateadas en la salida estándar

fork() creación de procesos. Duplica el proceso actual creando un proceso hijo con una copia exacta de su espacio de direcciones

retorna 0 al proceso hijo

retorna el pid del hijo (un número entero mayor a 0)

execv() / exec(): llamada al sistema que reemplaza la imagen de memoria del proceso actual por la de un nuevo programa ejecutable (por ejemplo ls o ps).

IMPORTANTE: si execv tiene éxito el código que este debajo NUNCA se ejecuta, ya que la sección de código e instrucciones del proceso fue completamente sobrescrita por el nuevo programa

exit (0): llamada al sistema que finaliza inmediatamente la ejecución del proceso actual devolviendo el código de retorno 0 al OS indicando éxito

main(void): función principal y punto de entrada obligatorio de cualquier programa ejecutable en C

pid_t: tipo de dato primitivo de C utilizado para almacenar identificadores de procesos (PIDs)

for (c = 0; c < 3; c++) un for de toda la vida que se ejecuta 3 veces

a. caso 1:

el proceso padre ejecuta el print imprimiendo "hola"

fork crea el proceso hijo

en el padre: x recibe el pid del hijo. la condición del if es falsa. salta el bloque del if y ejecuta el exit 0, finalizando el padre

en el hijo x recibe el valor 0. la condición es verdadera y entra al if

ahí el hijo ejecuta el exevc. Su espacio de código es reemplazado por el comando ls y se imprime el contenido del directorio

las líneas siguientes no se ejecutan porque exevc las cambia

b. caso 2: imprime hola

fork crea el proceso hijo, el hijo es 0 entonces entra al if y ejecuta exevc "ps". Reemplaza su memoria e imprime la lista de procesos activos, las líneas siguientes no se ejecutan

en el padre se cambia el proceso por el ls imprimiendo los directorios, el printf "fin" no se ejecuta

c. caso 3: imprime el mensaje

crea el proceso hijo con el pid 0, imprime el primer mensaje, luego los procesos activos y sale

el padre imprime como te fue y nada más

19. imprime comienzo con un salto de línea

el for se ejecuta 3 veces incrementado C

esto hace que el fork se duplique en cada iteración, siendo 8 en total

20. al ejecutar el fork el sistema realiza una copia independiente del espacio de memoria

del padre para el hijo, por lo tanto la variable p no se comparte, entonces se imprime 1 un total de 8 veces

para producir la impresion esperada la variable p debe estar dentro del for para copiar el valor antes de pasarselo al hijo, asi incrementariamos con la copia ya hecha

21 22 y 23 en papel

24. bus de direcciones: 32 bits

direccionamiento 1 byte por direccion

tamaño de pagina 2048 bytes = 2 KiB

proceso p1: requiere 51.358 bytes para codigo y 68131 bytes para datos

a. bit para pagina y desplazamiento

desplazamiento 2048 = 11 bits

bits de pagina $32 - 11 = 21$ bits

b. tamaño logico maximo de un proceso

2^{32} bytes = 4294967296

c. cantidad maxima de paginas que puede tener un proceso

$2^{21} = 2097152$

d. marcos disponibles

ram total / tamaño de marco = $4294967296 / 2048 = 2097152$ marcos

e. paginas necesarias para codigo p1

$51358 / 2048 = 25077 = 26$ paginas

f. paginas necesarias para datos

$68131 / 2048 = 34$

g. fragmentacion interna entre codigo y datos

NO puede hacer en una misma pagina C y D

$C(26 \times 2048) - 51358 = 53248 - 51358 = 1890$

$D(34 \times 2048) - 68131 = 69632 - 68131 = 1501$

h. 20 bits de pagina = $2^{20} = 1048576$

tamaño minimo en ram: si consideramos el estandar de pte = 4 bytes: $1048576 \times 4b = 4$ MiB

si tomamos la indicacion literal de 1KiB por entrada = $1048576 \times 1 \text{ KiB} = 1 \text{ GiB}$

i. nivel 1 posee $2^{10} = 1024$ entradas y siempre debe estar cargado en ram

nivel 2 posee $2^{10} = 1024$ entradas. P1 solo usa 60 paginas reales (26 + 34)
en RAM se instanciaran unicamente el directorio de paginas (1 tabla) mas las tablas de
2do nivel necesarias para sus 60
tamaño minimo en ram con PTE = 4 bytes
directorio 1: $1024 \times 4B = 4KiB$
tablas n2 activas= 1 a 2 x $4KiB = 4 KiB$ a $8KiB$
ocupacion total entre $8KiB$ y $12KiB$

j. entonces podemos concluir que la paginacion de 1 nivel desperdicia una gran cantidad de memoria ram al requerir reservar espacio para la tabla entera con muchas paginas vacias

la paginacion de 2 niveles logra un gran ahorro de memoria permitiendo instancias dinamicamente solo las tablas de segundo nivel que corresponden a las zonas de memoria virtual que el proceso efectivamente utiliza